Analyze the two GenAI programs. The analysis must include:

a. Name of two GenAI programs

Claude and Gemini

b. How you accessed them (e.g., Google query, CoPilot on my computer)

Google search

c. Prompt used. (You must use the same prompt for both GenAIs.)

Objective

Write a C program that picks a secret number between 1 and 10.

The user gets up to 3 tries to guess the number.

Requirements

- Use a fixed secret number (for example, 7).
- Use printf to prompt the user and scanf to read their guess.
- Use a while or for loop to let the user guess up to 3 times.
- Use if-else statements to check if the guess is too high, too low, or correct.
- Stop the loop immediately if the user guesses the correct number.
- Print a final win or lose message.

Example Output

text

```
Guess a number between 1 and 10.
Attempt 1/3. Enter your guess: 5
Too low! Try again.
Attempt 2/3. Enter your guess: 8
Too high! Try again.
Attempt 3/3. Enter your guess: 7
Correct! You win!
```

d. Generated code from both GenAIs.

Claude program

```c
#include <stdio.h>

int main(void) {
    int secret = 7;
    int guess;
    int attempts = 3;
    int won = 0;

    printf("Guess a number between 1 and 10.\n");

    for (int i = 1; i <= attempts; i++) {
        printf("Attempt %d/%d. Enter your guess: ", i, attempts);
        scanf("%d", &guess);

        if (guess < secret) {
            printf("Too low! Try again.\n");
        } else if (guess > secret) {
            printf("Too high! Try again.\n");
        } else {
            printf("Correct! You win!\n");
            won = 1;
            break;
        }
    }

    if (!won) {
        printf("Out of attempts! The number was %d. You lose.\n", secret);
    }

    return 0;
}
```

```c
#Gemini Program
#include <stdio.stdio.h>
#include <stdio.h>
```

```c
int main() {
    int secret = 7;
    int guess;
    int max_attempts = 3;
    int won = 0;

    printf("Guess a number between 1 and 10.\n");

    for (int attempt = 1; attempt <= max_attempts; attempt++) {
        printf("Attempt %d/%d. Enter your guess: ", attempt,
max_attempts);
        scanf("%d", &guess);

        if (guess == secret) {
            printf("Correct! You win!\n");
            won = 1;
            break; // Stop the loop immediately on correct guess
        } else if (guess < secret) {
            if (attempt < max_attempts) {
                printf("Too low! Try again.\n");
            } else {
                printf("Too low!\n");
            }
        } else {
            if (attempt < max_attempts) {
                printf("Too high! Try again.\n");
            } else {
                printf("Too high!\n");
            }
        }
    }

    if (!won) {
        printf("Sorry, you ran out of attempts! The secret number was
%d.\n", secret);
    }

    return 0;
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness: The Claude code works and follows the direction correctly. However it is missing a check for valid input, so if the user enters something random it would not run correctly. The Gemini code has a syntax error: “#include <stdio.stdio.h>” at the top, it is not only unnecessary (the correct library is included a line below) but it also messes up the compiler. It is also missing an input checker so it will run into the same problem as the first.

Execution time: Both programs use a for loop which is optimal for execution time. This is the only iteration and has the largest effect on execution time.

Space complexity: both programs have 4 int variables initialized at the top (secret, guess, attempt/max_attempt, and won), as well as an int counter variable used in the for loop (i and attempt). They also both use a for loop instead of a while loop (less memory). Ultimately, neither program is very memory intensive and they have effectively the same design and variable use.

Maintainability: The programs have very minimal comments (0 for Claude and 1 for Gemini) so documentation is rather poor. As stated before, they both lack any form of error handling which makes it less maintainable. However, variable names are all very logical and easy to follow and the program is very simple overall so the shortcomings are minimized.

f. Select one to use as the basis for this assignment and justify why you picked it.

Gemini because it has the most room for improvement.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Got rid of “#include <stdio.stdio.h>” and added comments for all major code blocks.
Added error handling so that inputs must be integers. Note that this does add a while loop, slightly increasing execution time but the tradeoff is worth it. This significantly increases maintainability. As for execution time and space complexity, there’s not much that can be done, variable use is relatively small, and for loops are already in use. If statements are necessary for all possible outcomes of the game (win, miss high, miss low, and loss). And now with the exception of the error handling block, that is the entire program.